FLASHINFER: EFFICIENT AND CUSTOMIZABLE ATTENTION ENGINE FOR LLM INFERENCE SERVING

presenter：Zhuoli Huang

11/4/2025

UCDAVIS
COMPUTER SCIENCE

Outline

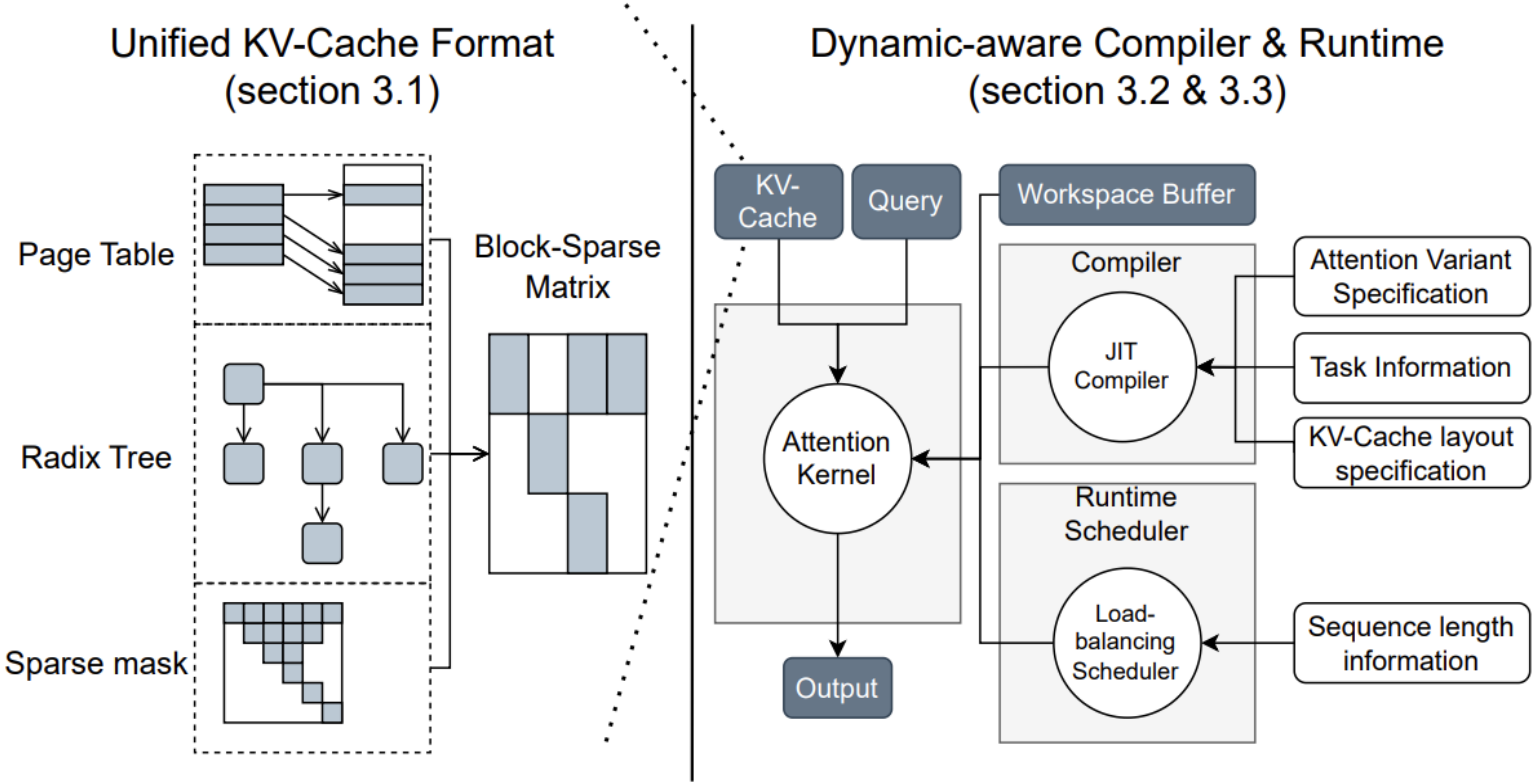
- Introduction
- Background
- Design
- Evaluation
- Related work
- Discussions
- Conclusion and future work

Introduction

- Transformer architecture as primary backbone for LLM with

Key feature attention mechanism: reads from the KV cache, computes outputs based on the current query.

- Challenges:
    1. LLM applications exhibit diverse workload patterns and input dynamics
    2. Modern hardware implementations necessitate the customization of attention operators
- **FlashInfer**: a code-generation based attention engine to accelerate attention computation
- Key design:
    1. FlashInfer utilizes a block-sparse format to tackle KVCache storage heterogeneity
    2. A customizable attention template supports different attention variants in FlashInfer.
    3. FlashInfer employs a dynamic load-balanced scheduling framework to handle input dynamism effectively.


Unified KV-Cache Format
(section 3.1)
Page Table
Radix Tree
Sparse mask
Block-Sparse
Matrix
Dynamic-aware Compiler & Runtime
(section 3.2 & 3.3)
KV-
Cache
Query
Workspace Buffer
Attention
Kernel
Output
Compiler
JIT
Compiler
Attention Variant
Specification
Task Information
KV-Cache layout
specification
Runtime
Scheduler
Load-
balancing
Scheduler
Sequence length
information

Introduction

Contributions:

1. Introduction of flexible block-sparse and composable formats addressing KV-Cache storage heterogeneity for efficient memory management and access
2. Development of a customizable attention template accommodating diverse attention variants, ensuring high performance execution via JIT compilation.
3. Design of a dynamic scheduling framework managing input dynamism while remaining compatible with CUDAGraph, maximizing hardware utilization.
4. Comprehensive evaluation demonstrating substantial improvements in kernel and end-to-end performance.

Background: FlashAttention

- an efficient algorithm for computing exact attention with reduced memory usage

- Advancements:

1. use online-softmax trick : update attention outputs on-the-fly using a constant amount of on-chip memory, to avoid materializing the attention matrix in GPU global memory
2. optimize loop ordering and pipeline design for Ampere and Hopper GPUs

$$O\left(\frac{1}{1/l_{qo}+1/l_{kv}}\right)$$

l_qo: the query
l_kv: key-value cache lengths

Background: Attention Composition

- Attention outputs for the same query and different keys/values can be composed by preserving both the attention outputs and their scales

$$\mathbf{LSE}(\mathcal{I}) = \log \sum_{i \in \mathcal{I}} \exp(\mathbf{q} \cdot \mathbf{k}_i)$$

$$\mathbf{O}(\mathcal{I}) = \sum_{i \in \mathcal{I}} \frac{\exp(\mathbf{q} \cdot \mathbf{k}_i)}{\exp(\mathbf{LSE}(\mathcal{I}))} \cdot \mathbf{v}_i$$

q: a query
I : an index set
k_i : the i-th key vector.

Attention State for I $\begin{bmatrix} \mathbf{O}(\mathcal{I}) \\ \mathbf{LSE}(\mathcal{I}) \end{bmatrix}$

$$\begin{bmatrix} \mathbf{O}(\mathcal{I} \cup \mathcal{J}) \\ \mathbf{LSE}(\mathcal{I} \cup \mathcal{J}) \end{bmatrix} = \begin{bmatrix} \mathbf{O}(\mathcal{I}) \\ \mathbf{LSE}(\mathcal{I}) \end{bmatrix} \oplus \begin{bmatrix} \mathbf{O}(\mathcal{J}) \\ \mathbf{LSE}(\mathcal{J}) \end{bmatrix}$$

$$= \begin{bmatrix} \frac{\exp(\mathbf{LSE}(\mathcal{I}))\mathbf{O}(\mathcal{I}) + \exp(\mathbf{LSE}(\mathcal{J}))\mathbf{O}(\mathcal{J})}{\exp(\mathbf{LSE}(\mathcal{I})) + \exp(\mathbf{LSE}(\mathcal{J}))} \\ \log(\exp(\mathbf{LSE}(\mathcal{I})) + \exp(\mathbf{LSE}(\mathcal{J}))) \end{bmatrix}$$

$\oplus$ can multiply sets of attention states can be composed in any order

offload partial-attention computations, thereby reducing memory usage and improving hardware efficiency

Background: Block/Vector Sparsity

- **Block Compressed Sparse Row (BSR):** a hardware efficient sparse format that groups non-zero elements into contiguous matrices of size (b_r, b_c)

    1. improves register reuse efficiency and demonstrates better compatibility with hardware matrix multiplication units on GPUs and NPUs
    2. provides the ability to skip empty blocks, reducing computational overhead

- Efficient utilization of the tensor core can be achieved with smaller block sizes, such as (16, 1) for matrix B in GEMM, or (1, 16) for matrix A (also known as **vector-sparse**) by first gathering rows/columns into contiguous shared memory and then applying dense tensor cores to these contiguous shared-memory data.

    beneficial for **fine-grained sparsity patterns**

- Flashinfer: support blocks with arbitrary column sizes B_c, offering greater flexibility and efficiency in handling diverse sparsity patterns

Design

- KV-Cache Storage
- Compute Abstraction
- Dynamism-Aware Runtime
- Programming Interface

Design: KV-Cache Storage

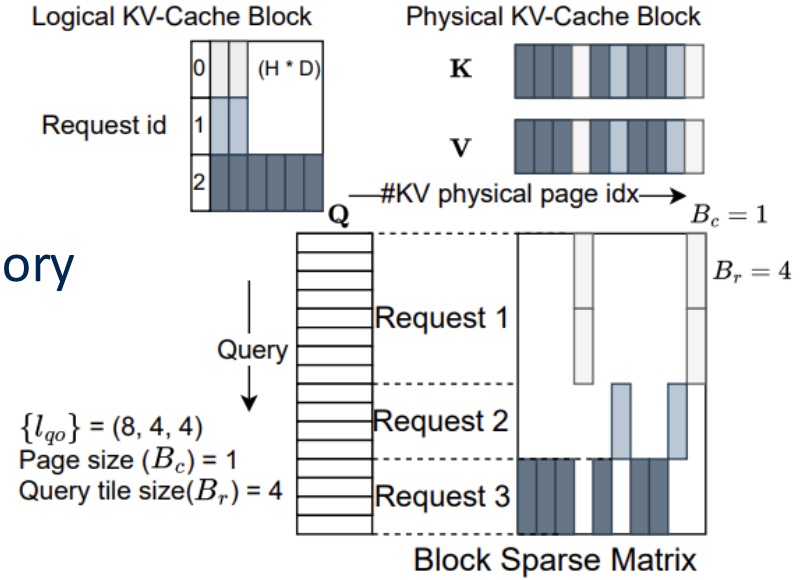
1.Block-Sparse Matrix as Unified Format

- Spilt KV-cache into many pages
- Non-contiguous memory storage with a minimum granularity of a block (or token) of (H, D) tensors (**a page**), to minimize memory fragmentation while enhancing memory reuse and cache hit rates

H : the number of heads

D : the hidden dimension

- Diverse data structures can be unified under a block sparse format



Design: KV-Cache Storage

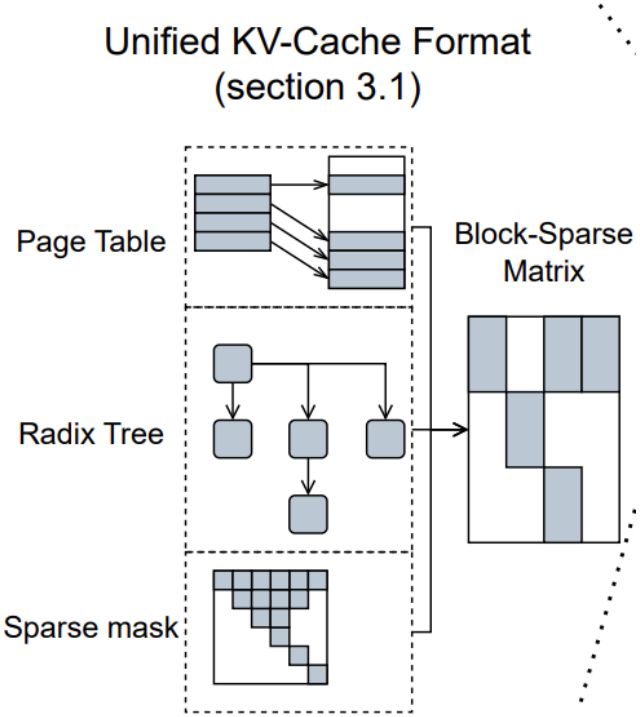
1.Block-Sparse Matrix as Unified Format

- Page Table ≈ Sparse Matrix

page-> different KV-cache block
non-zero block -> a certain query needs to access this KV page.

- Attention mechanisms ≈ Sparse Matrix

Abstract to which queries (rows) access which KV blocks (columns)
Tree(query) Mask(skip unimportant)



Design: KV-Cache Storage

1.Block-Sparse Matrix as Unified Format

- **Query , Output** -> ragged tensors without padding

Facilitates the compact packing of queries and outputs from diverse requests into a single tensor using index pointers to record the start and end positions of each request to ignore length

- **Key, Value->** ragged tensors without padding

Query, Key, and Value are all derived from the same input (the projection matrix W_q, W_k, W_v)

- **Store in BSR**

B_r : the query tile size

B_c : specified by KV-Cache management algorithms.

FlashInfer kernel implementations supports arbitrary (Br, Bc) values

Design: KV-Cache Storage

2.Composable Formats for Memory Efficiency

- **Multiple block sparse formats**

Using the fixed format "B_r" :

1. B_r is too large → many fragments
2. B_r is too small → cannot be reused and has poor performance
3. KV pages cannot be shared among different requests.

- **prefix -> submatrix**

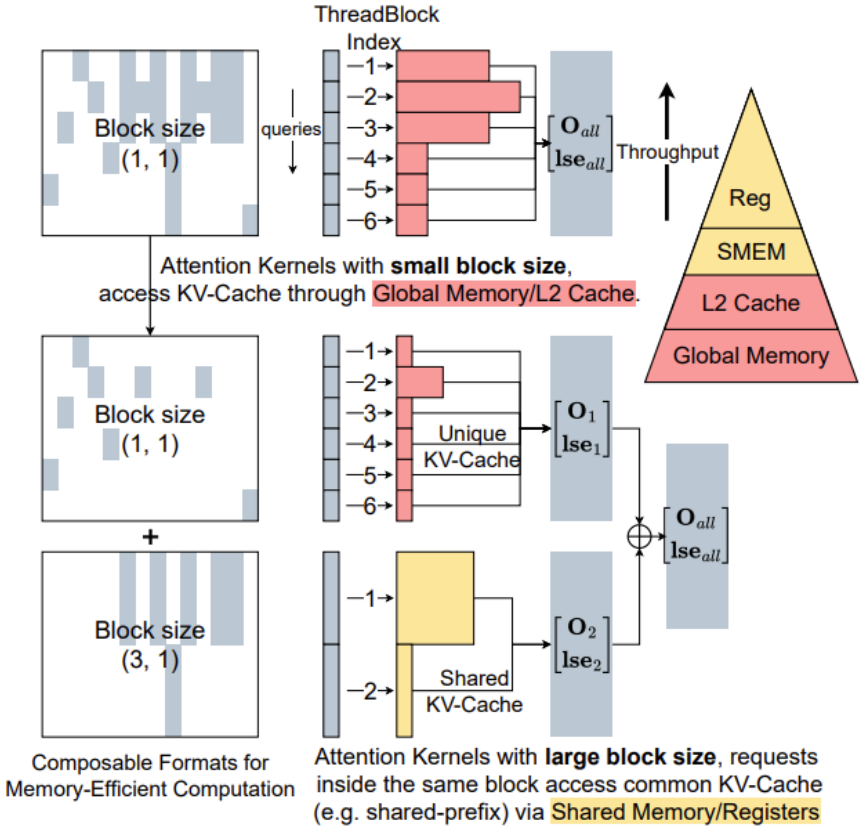
  prefix sharing -> dense submatrix -> larger B_r

No data removing; Access shared KV cache entries using fast shared memory and registers, significantly improving memory efficiency.

Design: KV-Cache Storage

2.Composable Formats for Memory Efficiency

- Higher memory efficiency and throughput rate

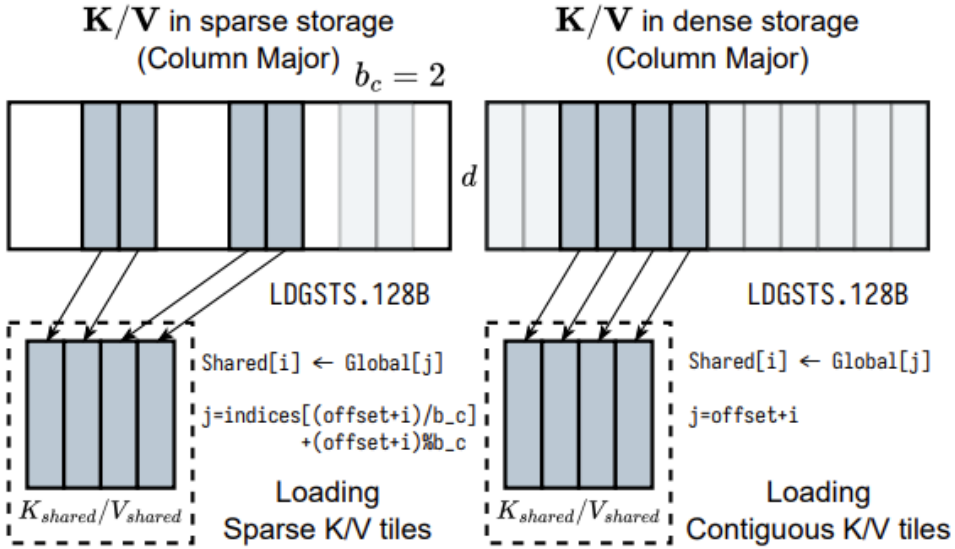


Design: Compute Abstraction

1.Global to Shared Memory Data Movement

- Blocks may not align with tensor core shapes
- Transferring tiles from scattered **global** memory to contiguous **shared** memory for dense tensor core operations

Sparse: indices arrays of the BSR matrix
Dense: row index affine transformations



Design: Compute Abstraction

1.Global to Shared Memory Data Movement

- last dimension of the KV-Cache remains contiguous-> maintaining coalesced memory access that fits GPU cache line sizes.

- Data loading method:

contiguous KV-Cache on Hopper GPUs: Tensor Memory Accelerator (TMA)(only for affine)

other: asynchronous copy instructions Load Global to Shared (Store) LDGSTS

- **Various** data loading modules-> **Consistent** kernel usage

Simplified implementation and maintenance

Design: Compute Abstraction

2.Microkernel with Different Tile Sizes

- Adapt to the varying operational intensities of LLM applications by the FlashAttention2 algorithm across multiple sizes.

(128, 64)-> (1, 16, 32, 64, 128) × (32, 64, 128)

- **Selecting method:**

workload intensity: minimal query tile size>= average query length

hardware resources: formulate register and shared memory constraints

Design: Compute Abstraction

3. JIT Compiler for Attention Variants

- At compile-time, the final tile size T_q to be used is determined. B_r =T_q

<16: CUDA Cores
>=16:Tensor Core
multiples of 64: Warp-Group Matrix Multiply-Accumulate (WGMMA)

- **Unified template design for custom attention variants**

a customizable CUDA template and a JIT compiler
Input: attention variant specification
Output: optimized kernel code

Design: Compute Abstraction

3. JIT Compiler for Attention Variants

- Variant Specification(functor)

1.Query Transform, **Key**Transform, **Value**Transform: before the attention computation
2.Output Transform: before returning
3.Logits Transform, LogitsMask: before the softmax computation, and the mask applied to the logits tensor.

Input: the kernel parameters, input(listed above) and current query/key/head index

Customized logits postprocessing:
1.custom mask 2.logits soft-cap 3. sliding window attention 4. using softmax or not

Query and **key** transformation functors can fuse normalization, RoPE, and projection into the attention kernel.

Attention Specification in Python

```
spec_decl = r"""
template <typename Params_, typename KernelTraits_>
struct FlashSigmoid {
  using Params = typename Params_;
  using KernelTraits = typename KernelTraits_;
  static constexpr bool use_softmax = false;
  float scale, bias;
  FlashSigmoid(const Params& params, int batch_idx, uint8_t* smem_ptr) {
    // Copy from CUDA constant memory to registers
    scale = params.scale;
    bias = params.bias;
  }
  ...
  float LogitsTransform(const Params& params, float logit_score, int batch_idx, int qo_idx, int kv_idx, int qo_head_idx, int kv_head_idx) {
    return 1. / (1. + expf(-(logits_score * scale + bias)));
  }
};
"""
attn_spec = AttentionSpec(
  "FlashSigmoid",
  dtype_q, dtype_kv, dtype_o, idtype, head_dim, is_sparse,
  additional_vars=[("scale", "float"), ("bias", "float")],
  additional_tensors=[],
  spec_decl=spec_decl
)
```

Part 1: Kernel Parameters Class

```
template <typename DTypeQ, typename DTypeKV, typename DTypeO, typename IdType>
struct Params {
  DTypeQ* q;
  DTypeKV* k, v;
  DTypeO* o;
  float* lse;
  IdType* qo_indptr, kv_indptr, kv_indices, kv_seq_lens;
  ...
  // (generated) additional vars
  float scale;
  float bias;
};
```

Part 4: Register custom operators in PyTorch

```
torch::Tensor attention_call(
  torch::Tensor q, torch::Tensor k, torch::Tensor v,
  ...
  float scale, float bias  // (generated) additional vars) {
  ...
  auto kernel = KernelTemplate<FlashSigmoid<Params<{{dtype_q}}, {{dtype_kv}}, {{dtype_o}}, {{idtype}}>, KernelTraits>>;
  ...
}
// Register torch custom ops
TORCH_LIBRARY_IMPL("FlashSigmoid", CUDA, m) {
  m.impl("run", &attention_call);
}
```

Part 2: Kernel Traits class

```
struct KernelTraits {
  static constexpr HEAD_DIM = {{head_dim}};
  static constexpr IS_SPARSE = {{is_sparse}};
  ...
};
```

Part 3: Kernel Body

```
template <typename AttentionSpec>
__global__ KernelTemplate(
  AttentionSpec::Params params) {
  ...
  // Init attention specification class.
  AttentionSpec attn(params, batch_idx, smem_ptr);
  ...
  // Iterate over all elements inside the thread logits tile.
  for (int i = 0; i < size(logits_tile); ++i) {
    // convert register index i to qo_idx, kv_idx, etc.
    qo_idx = get<0>(logits_tile(i));
    kv_idx = get<1>(logits_tile(i));
    ...
    logits_tile(i) = attn.LogitsTransform(
      params, logits_tile(i),
      batch_idx, qo_idx, kv_idx,
      qo_head_idx, kv_head_idx);
  }
  ...
}
```

Design: Dynamism-Aware Runtime

Load-balanced Scheduling Each time, the currently most idle CTA takes over the next heaviest chunk.

- Minimize Streaming Multiprocessor(SM) idle time by distributing the workload evenly across all SMs
- **Input:** the sequence length information of the query/output and key/value dimensions
- **Output:** the mapping between the workload and Cooperative Thread Arrays (CTAs) and the index mapping for partial and final outputs
- Generate deterministic aggregation order when provided with identical sequence length information.

Algorithm 1 FlashInfer's balanced scheduling algorithm

Input: $\{l_{qo}(i), l_{kv}(i)\}_i$, query tile size T_q.
Define the cost of a tile l_q, l_{kv} as (α, β are hyperparameters):

$$\text{cost}(l_q, l_{kv}) = \alpha l_q + \beta l_{kv}$$

Compute the maximum KV chunk size L_{kv} by

$$L_{kv} \leftarrow \frac{\sum_i \lceil \frac{l_{qo}(i)}{T_q} \rceil \cdot l_{kv}(i)}{\#\text{CTA}}$$

Split each query tile's KV into chunks, with maximum size L_{kv}, we assign each chunk a work index w, and the length of the chunk is $l_{kv}(w)$.
Let $W = \{(w, l_{kv}(w))\}$ and sort the entries in descending order of length.
$Q \leftarrow \text{PriorityQueue}(\{(c, 0)\})$ where c is the CTA index.
while $W \neq \emptyset$ **do**
  c, current_cost $\leftarrow Q.\text{pop}_{min}()$
  $w, l_{kv}(w) \leftarrow W.\text{pop}()$
  new_cost $\leftarrow$ current_cost + cost$(T_q, l_{kv}(w))$
  Assign chunk w to CTA c
  Q.push$((c,$ new_cost$))$
end while

Design: Dynamism-Aware Runtime

Load-balanced Scheduling information can be reused to amortize overhead

Step:

1.long KV are split into multiple chunks

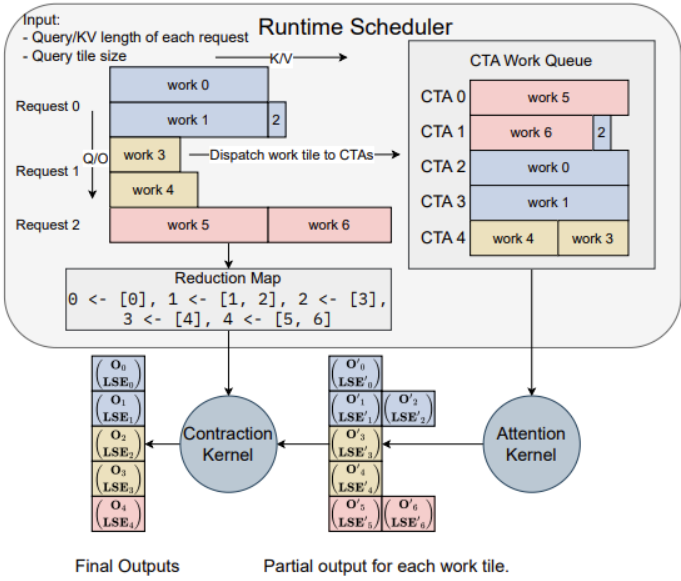
2.Assigned to CTA

3.Every work Attention Kernel

4.Reduction Map: How are each final output composed of various partial outputs?

5.Contraction Kernel

- The entire process is achieved through the **persistent kernel** (attention and contraction) and **fixed grid size**, achieving efficiency and determinism. It is fully compatible with CUDA Graphs and reduces the startup and synchronization overhead of the GPU.



Design: Programming Interface

- offers a programming interface designed for seamless integration with existing LLM serving frameworks

- Init(attn_spec,task_info,workspace)
- Kernel performs JIT compilation and caching(reuse)
- block size,tile size, KV ->various CUDA graph
- Runtime automatically selects the optimal Graph

- **plan(CPU):** Analyze the length of the dynamic sequence and generate a load balancing plan
- **run(GPU)**: outputting the attention results

```
# Create workspace buffer
workspace = torch.empty(...)
seqlen_info.init()

# Compile: create CUDAGraphs
graphs = []
for task_info in task_infos:
  # Init: compile kernels according to spec
  attn = AttentionWrapper(attn_spec, task_info, workspace)
  g = torch.cuda.CUDAGraph()
  # Dummy plan
  attn.plan(seqlen_info)
  # Capture CUDA graphs
  with torch.cuda.graph(g):
    for i, layer in enumerate(layers):
      ...
      attn.run(...)
      ...
  graphs.append(g)

# Runtime: select the best CUDAGraph
g = select_graph(graphs)
finished = False
# Text generation loop
while not finished:
  seqlen_info.update()
  # Plan per generation step
  attn.plan(seqlen_info)
  # Replay CUDA-Graph
  g.replay()
```

Evaluation

kernel-level and end-to-end performance

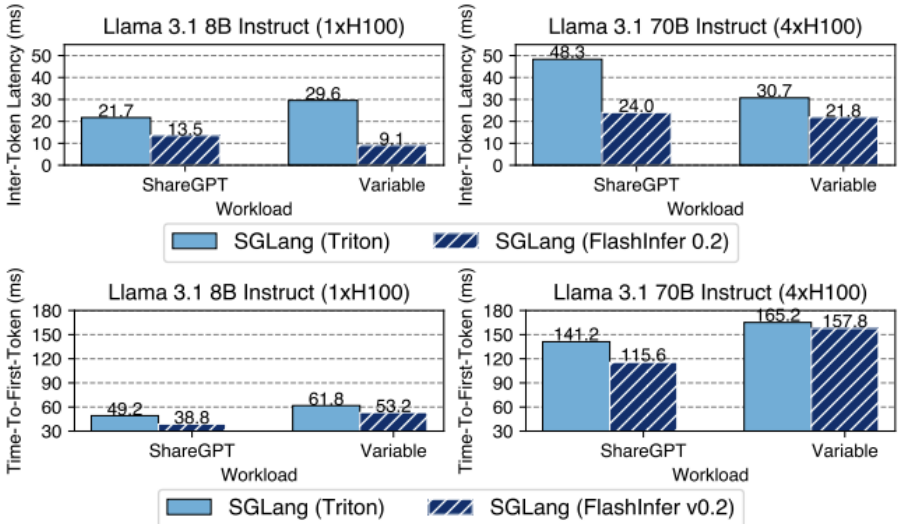
- LLM serving benchmark: 29-69% inter-token-latency reduction compared to Triton backend
- long-context inference:28-30% latency reduction
- LLM serving with parallel generation: 13-17% speedup

- Configurations: NVIDIA A100 40GB SXM and H100 80GB SXM GPUs
- CUDA 12.4 and PyTorch 2.4.0 and f16 precision for storage and computation

Evaluation

1.End-to-end LLM serving performance

- Triton: a leading LLM serving engine optimized for NVIDIA GPUs with closed-source attention kernels
- Dataset: sequence lengths between 512 and 2048 tokens
- 29-69% inter-token-latency reduction compared to Triton backend



Evaluation

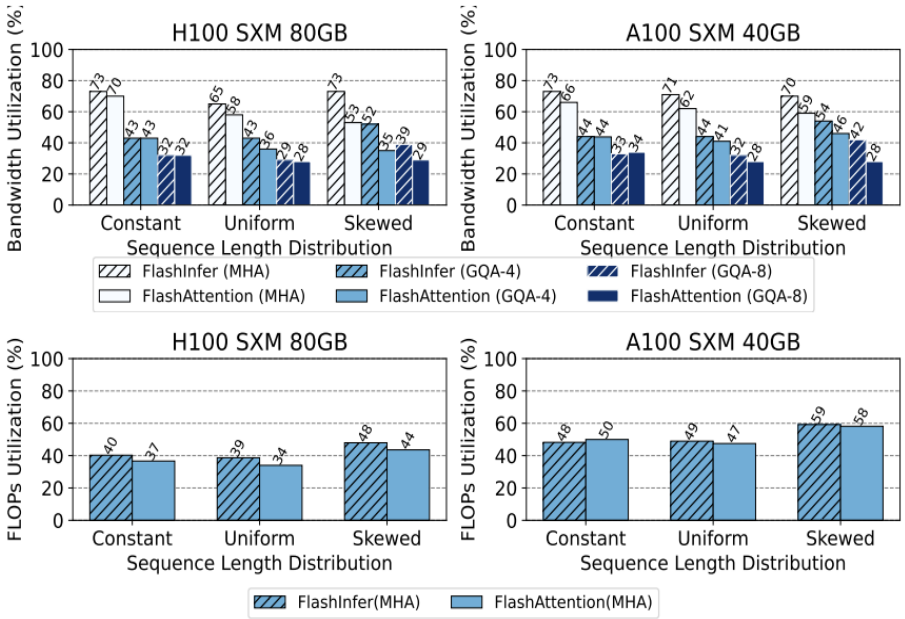
2.Kernel Performance for Input Dynamism

- Three different sequence length distributions:

constant (1024),
uniform (512 to 1024)
skewed (Zipf distribution with average length 1024)

- the higher the better

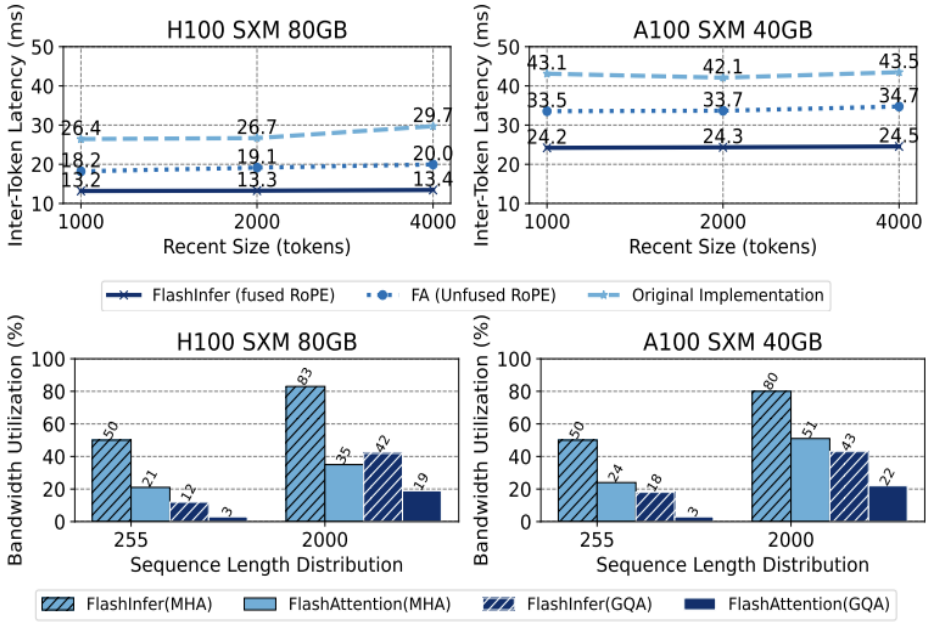
- Uniform, Skewed perform better
- Decode(top) attention outperforms better with versatile tile size selection



Evaluation

3. Customizability for Long-Context Inference

- Streaming-LLM: a recent algorithm capable of million-token inference with constant GPU memory usage need a fused kernel combining RoPE While only 20 lines code for Flashinfer
- 28-30% latency reduction
- 1.6-3.7x higher bandwidth utilization



importance of customizability of attention kernels

Evaluation

4. Parallel-Generation Performance

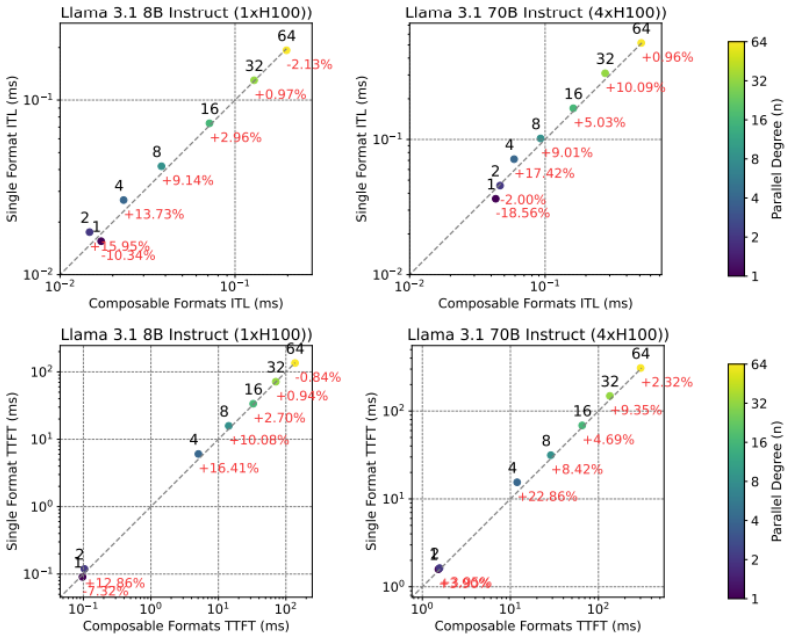
- Composable formats allow for the decoupling of attention computation between the shared prefix and the subsequent suffix to expedite parallel decoding
  - Llama 3.1 models with 8B and 70B
  - request rate :16
  - parallel tokens:1,2,4,8,16,32,64
  - above the diagonal line(y>x): composable formats> single format

4≤ n ≤ 32:speedups

n=4 Peak speedup

n<4:insufficient increase in block size

n>32:the computation ceases to be dominated by attention processes



Related work

- **Attention Optimizations**

FlashAttention v2/v3, LeanAttention

Flashinfer: Block-sparse + variable-length + load balancing

- **Sparse Optimizations on GPUs**

FusedMM, TC-GNN

Flashinfer: Arbitrary block sizes on Tensor Core

- **Attention Compilers**

FlexAttention, Mirage

Flashinfer: Native CUDA backend + extensible template

- **LLM Serving Systems**

PagedAttention, SGLang, vAttention

Flashinfer: Unified composable kernel for serving systems

Discussion

- **Current Scope:**
    1. Forward-only inference kernels
    2. Training requires backward template support

- **Design Advantages:**
    1. Decoupled computation & scheduling
    2. Runtime-scheduler enables dynamic load balancing
    3. Cross-architecture support (Ampere → Hopper)
    4. Functional form covers most attention types

Conclusion and future work

- A versatile and efficient attention engine for LLM serving
- A unified block-sparse storage and composable formats for memory efficiency, JIT compilation for customization and load-balanced scheduler for input dynamism
- Strong performance in kernel-level and end-to-end LLM serving metrics
- Future: explore compiling higher-level DSLs to attention specifications in FlashInfer, as well as code generation to other backends

Q&A

UCDAVIS
COMPUTER SCIENCE